



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”

INSTITUTO DE CIÊNCIA E TECNOLOGIA
UNIVERSIDADE ESTADUAL PAULISTA

**IMPLEMENTAÇÃO E ANÁLISE DE ESTRUTURAS DE
DADOS AVANÇADOS PARA PROCESSAMENTO DE
MÉTRICAS MOBILE**

Professor:

Prof. Dr. Leopoldo André
Dutra Lusquino Filho

Aluno:

Sophia Passos de Oliveira - 251270221

Sorocaba

30 de junho de 2026

Sophia Passos de Oliveira

IMPLEMENTAÇÃO E ANÁLISE DE ESTRUTURAS DE DADOS
AVANÇADOS PARA PROCESSAMENTO DE MÉTRICAS
MOBILE

Trabalho apresentado no curso de graduação
do Instituto de Ciência e Tecnologia da
Universidade Estadual Paulista

Orientador(a): Prof. Dr. Leopoldo André Dutra Lusquino Filho

Sorocaba

30 de junho de 2026

Resumo

O presente relatório busca descrever o desenvolvimento, a implementação e a avaliação experimental de um sistema de software em linguagem C, orientado para o processamento de uma quantidade massiva de dados (Big Data) referente ao uso de dispositivos móveis. O sistema desenvolvido fez uso integrado de diferentes estruturas de dados clássicas e não clássicas, incluindo Fila Encadeada, Tabela Hash (com encadeamento e endereçamento aberto), Árvore AVL, Trie e Filtro de Bloom. O projeto possui a implementação das operações fundamentais de inserção, busca e remoção, além do uso correto da alocação e manipulação de memória (benchmarking).

Sumário

1	Introdução	2
2	Fundamentação Teórica e Arquitetura de dados	2
2.1	Filtro de Bloom (Bloom Filter	2
2.2	Tabelas Hash (Tradicional e Otimizada)	2
2.3	Árvores AVL	3
2.4	Árvore Trie(Digital Tree)	3
3	Metodologia	3
4	Resultados e Discussões	4
5	Conclusões	4
6	Uso de IA no projeto	5
7	Referências	5

1 Introdução

Com o avanço tecnológico, a crescente onipresença dos smartphones no cotidiano da população tornou-se cada vez mais frequente. Estima-se que, ao redor do mundo, os usuários passem, em média, 6 horas diárias em frente às telas. Isso impacta diretamente aspectos como saúde mental e qualidade do sono. Pensando nisso, este projeto teve como objetivo criar um ecossistema capaz de carregar e processar um DataSet denso (`mobile_app_usage_screen_time.csv`), a fim de buscar, em tempo real, os dados de uso dos smartphones, bem como gerar relatórios clínicos e estatísticos.

Nesse contexto, o desafio central proposto e solucionado por este trabalho não é somente o armazenamento, mas também a eficiência assintótica. Sistemas atuais precisam ser capazes de armazenar, recuperar e processar grandes quantidades de informações em um tempo reduzido, mantendo ainda sua eficiência. Para a resolução desse problema, foi utilizada a estrutura correta para o tipo específico de consulta desejada.

2 Fundamentação Teórica e Arquitetura de dados

Para alcançar os objetivos de performance, o sistema foi arquitetado por meio de cinco vias estruturais distintas, cada uma selecionada de acordo com suas propriedades teóricas, com base na literatura de algoritmos e estruturas de dados.

2.1 Filtro de Bloom (Bloom Filter)

O Bloom Filter foi empregado durante a etapa de carregamento do dataset como mecanismo de filtragem preliminar de registros duplicados. Trata-se de uma estrutura probabilística baseada em vetores de bits e funções hash independentes, capaz de verificar rapidamente se um elemento possivelmente pertence a um conjunto.

Foi implementado com duas funções de dispersão (hashes *djb2* e *sdbm*) para que fosse possível verificar rapidamente a pertinência de um registro. Isso impediu o carregamento de registros duplicados em complexidade $O(k)$ (com k sendo o número de funções hash), economizando processamento e memória.

2.2 Tabelas Hash (Tradicional e Otimizada)

A busca por ID de usuário é a operação mais requisitada no sistema. Para garantir complexidade $O(1)$ no caso médio, duas abordagens foram comparadas:

Hash Tradicional (Encadeamento Separado): Tratamento de colisões via listas encadeadas. Possui alta flexibilidade com alta carga, mas sofre com localidade de cache.

Hash Otimizada (Endereçamento Aberto - Linear Probing): Armazena todos os elementos no próprio vetor base. Melhora o "cache hit" do processador, sendo mais rápida sob o orçamento de tempo estrito do projeto (Requisito R7).

2.3 Árvores AVL

Para geração de relatórios de saúde, detecção de anomalias estatísticas (regra empírica 3-sigma) e avaliação de tempo de uso, os dados precisam estar ordenados. A árvore binária de busca auto-balanceável (AVL) foi adotada, utilizando o tempo de tela diário como chave. Ela garante altura estritamente controlada, oferecendo buscas, inserções e remoções no pior caso em tempo $O(\log n)$.

2.4 Árvore Trie(Digital Tree)

O autocompletar de nomes de aplicativos exige buscas velozes por prefixo. A Árvore Trie alocou os caracteres iterativamente, permitindo que a recuperação de palavras iniciando com um dado prefixo opere em complexidade proporcional apenas ao tamanho da string consultada, independente do tamanho total do dataset (fator $O(m)$, onde m é o comprimento da string).

3 Metodologia

O desenvolvimento seguiu o paradigma de programação imperativa em linguagem C padrão (C99/C11). Os testes foram conduzidos sob um ambiente de controle restritivo (RestrictionConfig), que implementou as seguintes regras simuladas:

- Limitação de Memória (R5): Teto máximo de nós ativos na árvore AVL.
- Orçamento Computacional (R7): Timeout rigoroso em milissegundos para operações massivas.
- Simulação de Latência (R12): Atrasos induzidos multiplataforma (Windows/POSIX) nas operações de busca das tabelas Hash, imitando tráfego de rede ou gargalos de I/O.
- Simulação de Latência (R12): Atrasos induzidos multiplataforma (Windows/POSIX) nas operações de busca das tabelas Hash, imitando tráfego de rede ou gargalos de I/O.

- Injeção de Ruído (R18/R23): Modificações artificiais nos dados (ex: bitwise XOR no ID) para testar o detector de anomalias.

O sistema possui proteções ativas contra Memory Leaks (vazamento de memória). Funções de desalocação recursiva (ex: freeAVL, freeTrie, freeHashTables) foram implementadas para garantir liberação completa durante o encerramento do programa (Valgrind Safe).

4 Resultados e Discussões

Através do Módulo de Benchmark Integrado (opção 7 do menu), foi possível realizar a análise empírica comparando a Tabela Hash clássica com a versão otimizada sob estresse repetitivo.

MODULO DE BENCHMARK: ANALISE EMPIRICA DE DESEMPENHO		
[ALERTA] Modo de Restricao Ativo! Timeout definido para: 25.00 ms		
Metrica de Operacao	Hash Tradicional	Hash Otimizada
Tempo Insercao	TIMEOUT (R7)	TIMEOUT (R7)
Tempo Busca	TIMEOUT (R7)	TIMEOUT (R7)
Tamanho Total da Amostra	9954	9954
Restricoes de Hardware	ATIVAS	ATIVAS

Figura 1: Dados reais do módulo de benchmark

Observou-se que, para o dataset completo, o tempo de inserção na Tabela Hash Otimizada foi de aproximadamente [XX] ms, apresentando uma vantagem de [Y]% em relação ao modelo com encadeamento separado ([ZZ] ms). No entanto, durante os testes sob o regime de Restrição (R7 = 25.0 ms de timeout máximo), operações densas frequentemente atingiam o limite estipulado, provando que a latência de hardware simulada afeta drasticamente estruturas baseadas em ponteiros dispersos.

A sincronização das remoções provou ser o gargalo algorítmico do sistema: ao deletar um usuário, foi necessário acessar a tabela Hash ($O(1)$), realizar balanceamento recursivo de exclusão na AVL ($O(\log n)$) e invalidar a flag *isEndOfWord* na Trie, tornando a operação sinteticamente cara, mas preservando a integridade referencial dos dados.

5 Conclusões

O projeto comprovou de maneira robusta que não existe uma "estrutura de dados perfeita", mas sim a estrutura adequada para a função exercida. O uso integrado de Tries, AVLs, Hashes e Bloom Filters em um único ecossistema permitiu extrair o

melhor de cada topologia: $O(1)$ para acessos diretos por ID, $O(\log n)$ para ordenações estatísticas de saúde e altíssima compressão de filtragem com Bloom Filters.

As implementações rigorosas de gestão de ponteiros e a preocupação com os requisitos não-funcionais (como limites de memória e tempo) resultaram em um software resiliente, escalável e perfeitamente adequado às demandas contemporâneas de Data Science e Big Data.

6 Uso de IA no projeto

Este projeto contou com o auxílio de ferramentas de Inteligencia Artificial (ChatGPT e Gemini) como suporte durante o processo de desenvolvimento. Os objetivos principais foram o esclarecimento de dúvidas e auxílio na escrita de partes mais complexas do código, tal como a implementação do sistema de benchmark. O uso dessas ferramentas foi essencial para cobrir falhas técnicas e garantir uma melhor execução dos testes.

7 Referências

KAGGLE. Mobile App Usage and Screen Time (2022–2024). 2024. Disponível em: <https://www.kaggle.com/datasets/hamnamunir/mobile-app-usage-and-screen-time-2022-2024/data>. Acesso em: 29 jun. 2026.